

Chapter 5 | Exercise Answers

The Relational Data Model and Constraints

Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, 7th edition.

Questions 5.1-5.20 follow the photographed chapter supplied for this course. Use the question numbers and printed page locators to read these answers beside your textbook.

These are original worked answers, not an official publisher solution manual. Open-ended designs state their assumptions. Full question text and photographed textbook figures are not reproduced.

Examples are identified as textbook data or original test data. SQL output tables are produced by executing the shown queries. SQLite-specific limitations are stated where they affect the result.

Publication of these answers does not expand the assigned sections or examination scope. Only taught and assigned material is assessed.

5.1. Relational vocabulary

Source: photographed textbook, printed p./pp. 200; explanations 181-191.

Term	Meaning / example
Domain	Permitted atomic values, such as nonnegative integer credit hours.
Attribute	A named role for a value, such as Credit_hours.
N-tuple	One row with n attribute values; n is the number of attributes.
Relation schema	The relation name, attributes and their domains.
Relation state	The current set of tuples.
Degree	Number of attributes, not number of rows.
Relational database schema	The collection of relation schemas and integrity constraints.
Relational database state	The current relation states together; a valid state satisfies the constraints.

StudentId	Name	Major
S1	Alex	CS
S2	Blair	MATH

This original relation has degree 3 and two tuples. The allowed values and uniqueness rules belong to its schema, not just this displayed sample.

5.2. Does tuple order matter?

Source: photographed textbook, printed p./pp. 200; explanation 184.

Display A	Display B
S1, Alex	S2, Blair
S2, Blair	S1, Alex

No. Both displays represent the same set of tuples. Physical storage or a query result may have an order, but the relation has no intrinsic first or last tuple. Request ORDER BY when a SQL display must be sorted.

5.3. Why a relation has no duplicate tuples

Source: photographed textbook, printed p./pp. 200; explanations 184, 188.

A relation is a set, so inserting an identical tuple does not create a second distinct element. Two real-world events with otherwise identical values need an identifying attribute if both must be represented.

Representation	Two identical displayed rows
Mathematical relation	One set element.
SQL table without a suitable key	Duplicates may be stored; SQL commonly uses multiset semantics.
SQL SELECT DISTINCT	Duplicate result rows are eliminated.

Duplicate values in one column are not duplicate tuples. Two students may share a name while their StudentId values differ.

5.4. Key versus superkey

Source: photographed textbook, printed p./pp. 200; explanations 188-190.

Attributes	Superkey?	Candidate key?
StudentId	Yes, by the numbering rule	Yes
StudentId + Name	Yes	No: Name can be removed
Name	Not guaranteed	No

A candidate key is a minimal superkey: no attribute can be removed while preserving uniqueness for every valid state. Minimal does not mean that all candidate keys must have the same number of attributes.

5.5. Why designate a primary key?

Source: photographed textbook, printed p./pp. 200; explanations 189-190.

Choose one candidate key as the main identifier used by references and applications. Keep the other candidate-key constraints; selecting a primary key does not make the alternatives stop being unique.

Candidate	Possible role
StudentId	Primary key: compact and stable.
InstitutionalEmail	Alternate candidate key only if the institution guarantees uniqueness and presence.

A SQL UNIQUE column that permits NULL is not automatically a candidate key in the non-NULL relational sense. Declare the necessary NOT NULL rule too.

5.6. Characteristics of relations

Source: photographed textbook, printed p./pp. 200; explanations 183-186.

Property	Consequence
No tuple ordering	Reordering rows leaves the relation unchanged.
No duplicate tuples	Whole rows are distinct as set elements.
Attribute-value correspondence	Reordering columns is harmless only if each value stays associated with its attribute.
Atomic values in the basic model	Do not store a repeating list of phone numbers as one relational value in this basic model.
Domain membership	Each non-NULL value must have the appropriate domain.
NULL when permitted	Missing or inapplicable values require special treatment.

The textbook first uses ordered tuples matching an ordered schema, then gives an attribute-name mapping interpretation. Do not swap raw positional values without swapping their attribute definitions.

5.7. What NULL does and does not say

Source: photographed textbook, printed p./pp. 200; explanations 185-186.

Situation	Office phone
Has no office	NULL may mean not applicable.
Has an office but number is unknown	NULL may mean unknown.
Number exists but has not been supplied	NULL may mean unavailable.

SQL NULL does not itself distinguish these reasons. Use a separate status attribute if the distinction matters. NULL is not zero, an empty string, or proof that two people have the same unknown phone number.

5.8. Entity integrity and referential integrity

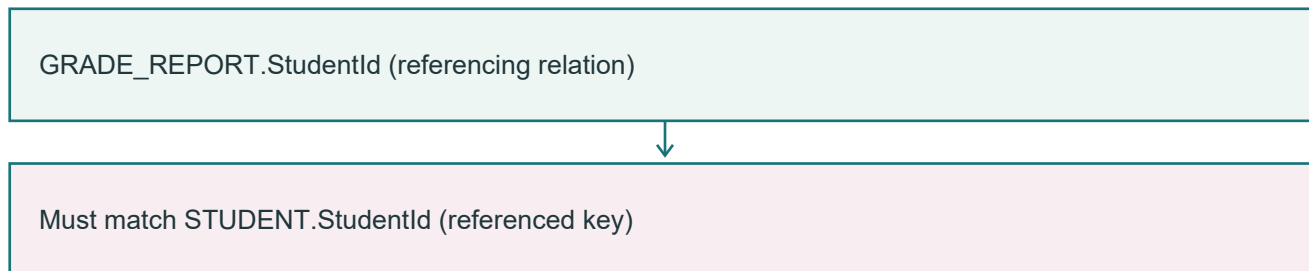
Source: photographed textbook, printed p./pp. 200; explanations 193-194.

Rule	Valid original example	Invalid example
Entity integrity	STUDENT key S1 identifies one row.	A primary-key component is NULL.
Referential integrity	GRADE_REPORT.StudentId S1 refers to existing STUDENT S1.	It refers to S9 when no S9 exists.

Entity integrity keeps each tuple identifiable. Referential integrity keeps references meaningful. Neither alone prevents every business error, such as recording an impossible grade or exceeding room capacity.

5.9. Defining a foreign key

Source: photographed textbook, printed p./pp. 200; explanations 193-194.

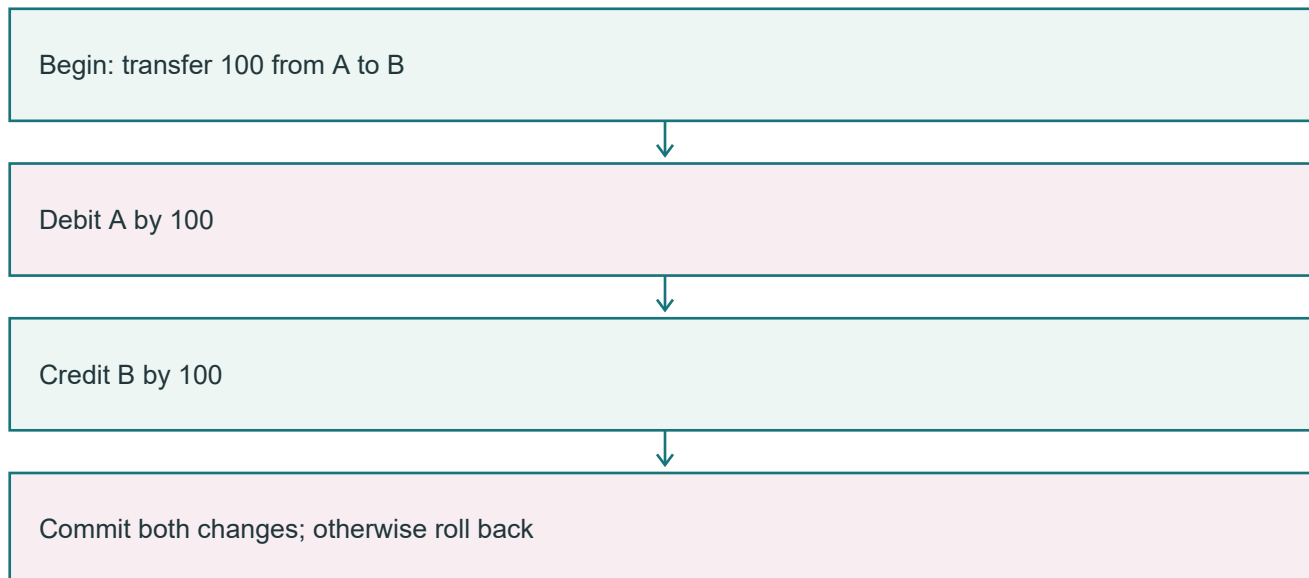


A foreign key is an attribute list whose values refer to a candidate key of a relation, possibly the same relation. The corresponding domains must be compatible. Repeated foreign-key values are normal: many grades may refer to one student.

A permitted missing reference can be represented by NULL. For composite SQL foreign keys, NULL rules depend on the matching rule and DBMS. In SQLite, any NULL component bypasses the parent-row match; declare every component NOT NULL when the reference is required.

5.10. A transaction is a logical unit

Source: photographed textbook, printed p./pp. 200; explanations 195-199.



A database transaction groups the operations of one logical task. It may contain one statement or many. The transfer must not finish with only the debit recorded. Integrity may need to be checked at the appropriate statement or transaction boundary. This example explains the distinction; it is not a complete banking implementation.

5.11. Check each proposed COMPANY update

Source: photographed textbook, printed p./pp. 200-201; Figure 5.6 p.192.

Each letter starts from the original Figure 5.6 state, not the previous letter's result. The results below were checked with foreign keys enabled and rejecting actions by default. SQL statements appear again in Answer 6.11.

Part	Result	Reason / repair
a	Rejected	Missing supervisor 222244555; insert the real supervisor first or correct the reference.
b	Accepted	One project added. Key 4 is unused; department 4 exists.
c	Accepted	One department added. The manager exists; manager membership in that same department is not a stated FK.
d	Rejected	Both employee 777624972 and project 15 are missing.
e	Accepted	One dependent added. Bdate is not part of the key and is nullable here.
f	Accepted	One row removed: Abner.
g	Rejected	Referenced by four employees, three projects and three department locations.
h	Accepted	Three assignments removed; the project itself remains.
i	Accepted	Three projects change. Department 1 exists.
j	Accepted	One row matches; Borg already has no supervisor, so stored values do not change.
k	Rejected	Three WORKS_ON rows refer to 30. An approved ON UPDATE CASCADE would change their Pno to 40.

For (g), preserve history by rejecting the deletion or explicitly reassigning affected employees/projects and handling locations in one authorized transaction. Cascading through all employee records is not an automatic business decision.

For (c), Chapter 5 does not declare a 15-character Dname limit. Applying Figure 6.1's VARCHAR(15) in a DBMS that enforces it would additionally reject the longer department name. SQLite does not enforce VARCHAR length without a CHECK.

The fixture retains Borg's NULL Hours from Figure 5.6. Figure 6.1 later marks Hours NOT NULL, which conflicts with that state; this fixture is not presented as a verbatim implementation of Figure 6.1.

5.12. Inserting airline data and checking references

Source: photographed textbook, printed p./pp. 201-202; Figure 5.8 p.202.

(a) Find or create the scheduled FLIGHT and FLIGHT_LEG, select an existing airplane, confirm both airports and the airplane type's landing permissions, then insert the dated LEG_INSTANCE. Initialize available seats from the actual plane and reservations under the chosen booking policy. Do the change atomically.

Part (b): check for a new instance	What to verify
Key	Flight_number + Leg_number + Date is new and non-NULL.
References	Scheduled leg, airplane and both actual airports exist.
Seat capacity	Available seats are nonnegative and no greater than the actual plane's capacity; reservations must agree with the chosen accounting rule.
Landing / scheduling	Airplane type can land at both actual airports; flight operates on that date; airplane assignments do not overlap.
Time	Arrival follows departure using dates and time zones, not only local clock values.

(c) For a scheduled FLIGHT_LEG, check the composite key, existing FLIGHT, both airports, valid leg number and schedule. No assigned airplane, dated instance or reservation is required merely to define a leg. The photographed final word "not" is unclear; this distinction states both the necessary checks and those that belong to an instance instead.

Part (d): referencing attributes	Referenced key
FLIGHT_LEG.Flight_number	FLIGHT.Flight_number
FLIGHT_LEG.Departure_airport_code	AIRPORT.Airport_code
FLIGHT_LEG.Arrival_airport_code	AIRPORT.Airport_code
LEG_INSTANCE.(Flight_number, Leg_number)	FLIGHT_LEG.(Flight_number, Leg_number)
LEG_INSTANCE.Airplane_id	AIRPLANE.Airplane_id
LEG_INSTANCE.Departure_airport_code	AIRPORT.Airport_code
LEG_INSTANCE.Arrival_airport_code	AIRPORT.Airport_code
FARE.Flight_number	FLIGHT.Flight_number
AIRPLANE.Airplane_type	AIRPLANE_TYPE.Airplane_type_name
CAN_LAND.Airplane_type_name	AIRPLANE_TYPE.Airplane_type_name
CAN_LAND.Airport_code	AIRPORT.Airport_code
SEAT_RESERVATION.(Flight_number, Leg_number, Date)	LEG_INSTANCE.(Flight_number, Leg_number, Date)

Landing permission, capacity and schedule consistency are additional business constraints, not all consequences of these twelve foreign keys.

5.13. A key needs a rule, not a promising name

Source: photographed textbook, printed p./pp. 201.

The question mentions `Product_section#` but the displayed schema has `Product#` and `Production_Section#`. No attribute named `Product_section#` is defined. Therefore a unique key cannot be established from the wording alone.

Explicit interpretation	Possible candidate key	Limit
One summary per product and production section	<code>Product#</code> + <code>Production_Section#</code>	Invalid if several stages or runs create several rows.
One summary per product, section and stage	<code>Product#</code> + <code>Production_Section#</code> + <code>Stage</code>	Still invalid if multiple production runs share the same stage.
One run per machine and precise start time	<code>Machine#</code> + <code>Machine_Time</code>	Only if <code>Machine_Time</code> is a start timestamp and no two runs start together on that machine.

`Product#` alone is not enough when a product crosses sections. Supervisor, warehouse and manufactured quantity can repeat. Ask which row represents one production event before declaring a primary key. These are conditional alternatives, not three proven keys.

5.14. Car-rental references and additional rules

Source: photographed textbook, printed p./pp. 202-203.

Foreign key	Referenced key
RENTAL.Cust#	CUSTOMER.Cust#
RENTAL_CAR.Rental#	RENTAL.Rental#
RENTAL_CAR.Car#	CAR.Car#
RENTAL_CAR.Driver#	DRIVER.Driver#
SERVICING.Garage#	GARAGE.Garage#
SERVICING.Car#	CAR.Car#

Assumptions for a workable key design: one car appears at most once per rental, giving RENTAL_CAR key (Rental#, Car#); one service record per garage, car and date gives SERVICING key (Garage#, Car#, Service_date). Repeated same-day services need another identifier.

Additional rule	Why ordinary foreign keys are not enough
No overlapping car or driver assignments	A valid car/driver may still be booked twice at the same time.
End after start; charges nonnegative	References say nothing about durations or money.
Rental date/time meaning is explicit	Reservation date and service date are different facts.

Driver# is not functionally determined by Car# across all rentals. Different drivers may drive the same car on different bookings. A complete SQL implementation of the stated key/FK assumptions appears in Answer 6.5.

5.15. Doctor visits and expenses

Source: photographed textbook, printed p./pp. 203.

Foreign key	Referenced key
VISIT.DSsn	DOCTORS.DSsn
EXPENSE.Visit_id	VISIT.Visit_id

Assume Visit_id uniquely identifies a visit; DSsn identifies a doctor. An expense references one visit. If an account may have several expense lines for the same visit, (Visit_id, Account#) is not sufficient as an expense key.

From_hospital, To_hospital and Account# cannot reference tables that are not supplied. HOSPITAL and ACCOUNT relations may be added under a revised design, but are not foreign keys to an existing relation in this question.

Return_date must not precede Departure_date. The requirement that every visit have at least one expense is not enforced by the expense-to-visit foreign key; it is a separate minimum-participation rule.

5.16. Users, borrowed books and supplies

Source: photographed textbook, printed p./pp. 203.

Reference	Meaning
BOOK_BORROWED.Book_isbn -> BOOK.Book_isbn	Borrowed book exists.
BOOK_BORROWED.(Ssn, Card#) -> USER.(Ssn, Card#)	The card belongs to this same user; declare a unique parent pair.
SUPPLY.Book_isbn -> BOOK.Book_isbn	Supplied title exists.
SUPPLY.SSsn -> SUPPLIER.SSsn	Supplier exists.

USER.Ssn	USER.Card#
U1	C10
U2	C20

Proposed borrowing Ssn	Card#	Result
U1	C10	Valid user/card pair
U1	C20	Invalid combination, although U1 and C20 each exist

Two independent existence checks would accept the wrong combination. An ordinary Ssn foreign key is implied by the composite reference; do not confuse card existence with ownership. If cards are globally unique, enforce that additional rule.

Repeated borrowing requires a date or event identifier in its key. Return_date may be NULL for an open loan and must not precede Issue_date when present.

5.17. Apartment bookings: valid and invalid inserts

Source: photographed textbook, printed p./pp. 203.

Foreign key	Referenced key
OPTION.Apartment#	APARTMENT.Apartment#
BOOKING.Agent_id	AGENT.Agent_id
BOOKING.Apartment#	APARTMENT.Apartment#

Original sample state, not textbook data. Test each proposed insertion independently, starting from this state:

AGENT: Agent id	Name	Phone
A1	Alex	555-0101
A3	Casey	555-0103

APARTMENT: Apartment#	Model	Address	Price perSquareFt
P1	Studio	1 Park Road	100
P2	Family	2 Park Road	120

OPTION: Apartment#	Option name	Extra price
P1	Balcony	500
P2	Storage	300

BOOKING: Agent id	Apartment#	Date	Booking price
A1	P1	2026-09-01	95000
A3	P2	2026-09-02	150000

Operation	Outcome
Insert BOOKING(A1, P1, 2026-09-20, 100000)	Accepted: both references exist.
Insert BOOKING(A9, P1, 2026-09-20, 100000)	Rejected: A9 does not exist.
Insert AGENT(A2, Blair, 555-0102)	Accepted: new key.
Insert another AGENT with Agent_id A1	Rejected: duplicate key, not a foreign-key violation.

AGENT has no outgoing foreign key in the supplied schema. Therefore no insertion into AGENT can violate a declared referential constraint in this design. This is a problem with that part of the question's premise, not a reason to invent an unrelated parent table.

Assume at most one booking for an apartment on a given date if (Apartment#, Date) is used as the key. Otherwise a BookingId or a more precise event identifier is needed.

5.18. When to split a phone number

Source: photographed textbook, printed p./pp. 203-204; Figure 5.1 p.183.

Split country code, area code and local number when the application must validate, route, group or search those components. Keep the components as strings to preserve leading zeros and formatting rules.

Need	Reasonable representation
Only display or dial a normalized complete number	One normalized phone string, plus extension when needed.
Count contacts by country code	Separate CountryCode and national-number components.

A code is not proof of a person's current physical location. Number portability and international formats also make a fixed area-code length unsafe.

5.19. Phone, name and repeating-value alternatives

Source: photographed textbook, printed p./pp. 204.

Part	Answer
(a) Missing phone components	Local_phone and Cell_phone each need their own area code for calling across regions; include the country code for international calls. Do not infer either code from the student's address, since the two numbers may come from different regions.
(b) Keep or split a phone	Store each complete normalized number in Local_phone / Cell_phone when only full-number use is needed; add separate code attributes when component queries or validation are required.
(c) Split a name	Given_name, Middle_name and Family_name can support sorting or addressing. Middle_name may be absent; names do not universally follow this pattern. Preserve a full display name when needed.
(d) Choice depends on use	Component searching and sorting favor components; simple display and culturally varied names favor a preserved display form. Neither decomposition is universally correct.
(e) Zero to five phones	Either five optional phone columns, or the child relation below. The latter avoids repeating column definitions.

STUDENT PHONE: StudentId	Slot	Phone
S1	1	555-0101
S1	2	555-0102

Use key (StudentId, Slot), a required foreign key to STUDENT, and CHECK(Slot BETWEEN 1 AND 5) with integer Slot. The key and slot domain limit each student to five rows. A student with zero phones has no child rows. Optionally forbid duplicate phone values per student if that is the intended rule.

5.20. Surrogate identifiers and changing names

Source: photographed textbook, printed p./pp. 204.

Part	Answer
(a) Natural identifier	A verified institutional or national identifier can be a candidate only under a stated uniqueness, presence and permitted-use rule. Name, birth date or phone alone is not guaranteed unique. The supplied personal attributes do not establish a universal natural key.
(b) Name change	If a name-based key is used, check the new key for conflicts and update every reference atomically. Appending a unique assigned discriminator can distinguish names, but then part of the identifier is artificial. A stable student number avoids name changes propagating through keys.
(c) Surrogate key	A generated identifier is compact and stable and need not expose personal data. It adds an attribute and does not prevent duplicate real-world people unless separate business uniqueness checks are also retained.

Example: StudentId S1 stays S1 when Alex Chen changes their display name. Two rows S1 and S2 can still represent the same person accidentally. A surrogate key does not solve identity verification.